

Assignment 1: Simplest Client-Server

1. Get Set Up

If you are not already using gitlab or github, you are now.

Create two Git repos, for your Server and Client projects. Name them TicTacToeClient and TicTacToeServer since tic tac toe is the final boss in this course.

Using Git Desktop (or cmd line), clone your repos so that you have access to two project folders being managed by Git.

Create a Unity project in both of your git repo folders.

In both projects, go to Window->Package Manager, in "Packages:Unity Registry" search for "Unity Transport" and install.

Get the simplest server and client components from:

<https://gitlab.com/netcode-in-games-campaign-manual/simplestclient/-/blob/main/Assets/NetworkClient.cs>

<https://gitlab.com/netcode-in-games-campaign-manual/simplestserver/-/blob/main/Assets/NetworkServer.cs>

Add the NetworkClient script to your client project. Add the NetworkServer into your server project.

In each project, create a gameobject and attach the appropriate network script to it.

Push an initial commit on both of your projects.

Link me as a collaborator on both of your projects.

<https://gitlab.com/FernandoTeaches>

<https://github.com/WindJesterFernando>

2. Initial Demonstration

Open NetworkClient.cs, find the string constant named IPAdrees and then edit it to your local IP Address. To discover your local IP, on Windows, use ipconfig in the windows terminal or, if you are on Mac, via your apple icon -> Sys Pref -> Network menu options.

Open your server program and run it. Now run your client program. Does it connect (and stay connected)? When you focus on your client program and hit the 'a' key, do you get a hello from your client in your server debug log?

Is there a delay in your debug log msgs? Do they not show up until you select the project? By default unity does not run in the background. To change this, go to Project Settings -> Player -> Resolution and Presentation and then toggle on the Run in Background option.

3. The "out" Parameter Modifier

What is the "out" keyword parameter modifier in C#? What does it do?

Why do we need to use the out keyword parameter mod with int values but not when we pass class instances?

The "out" keyword in C# allows methods to return multiple values by requiring the method to assign a value to the out parameter before returning. For value types like int, this is essential because they are passed by value, meaning changes inside the method don't affect the original variable. In contrast, class instances are reference types and can be modified directly by methods without needing the out keyword, since they are passed by reference by default.

4. Reading Code

The cornerstone of this course is the code contained with NetworkClient.cs and NetworkServer.cs. You are being challenged not just to understand the gist of it, but instead, seek to understand every line of it.

Reading code is a skill to get good at in and of itself. As you may have intuitively deduced, contrary to the way we read a book, the process of reading code is not as linear. Thus the question opens, how does one best read through code? Consider the following approach which is optimized to reduce cognitive overload, stress.

1. Scan through the code, look for any keywords or syntax that you do not understand. These are easily isolated and can initially be figured out. Answer the following: what keywords/syntax are not immediately known to you and (after some research) what do they do?

NativeArray	A fixed-size, contiguous memory array in Unity, designed for high-performance applications. It stores data in native memory, enhancing performance in multithreaded and parallel jobs, especially with the job system or burst compiler. Unlike managed arrays, NativeArray requires explicit allocation and disposal to manage memory properly.
ushort	This data type declares variables that store unsigned 16-bit (2-byte) integers, with a range of 0 to 65,525. It is an alias for System.UInt16 in the .NET framework.
NativeList	A dynamic, resizable collection in Unity designed for the job system and burst compiler, stored in native memory for better performance and lower overhead compared to managed collections like List<T>. It requires manual disposal to avoid memory leaks.
ScheduleUpdate	Is a method from Unity's Transport package. It schedules an update for the network driver. This method should be called once per frame to ensure the network driver processes any pending network events. It returns a JobHandle, which you can use to check if the update job is complete.
PopNetworkEventAndCheckForData	Is a custom method in your code that wraps around the PopEvent method from Unity's Transport package. The PopEvent method is used to receive the next available network event for a specific connection. It returns the type of the event and, if the event is of type Data, it also provides a DataStreamReader containing the received data. Your custom method checks if there is any data associated with the event and returns a boolean indicating whether there was an event to process ² .

2. Scan through the code and look for external classes, structures, enumerations and function calls. Read the documentation on them. This should take a while. Answer for each of the following:

2.1. What is the NativeList struct?

- a dynamic and resizable collection in Unity, designed for use with Unity's job system and burst compiler, that operates in native memory for better performance and lower overhead.

2.2. What is the `NetworkDriver` struct?

- Is the main API for interacting with Unity's transport package, functioning like a socket with extra features.

2.3. What is the `NetworkConnection` struct?

- Represents a connection obtained by calling `Accept()` on servers or `Connect(NetworkEndpoint)` on clients.

2.4. What is the `NetworkPipeline` struct?

- Is the identifier for a network pipeline obtained with `CreatePipeline(Type[])`.

2.5. What is the `FragmentationPipelineStage` struct?

- Is a pipeline stage used to fragment large packets into MTU-sized chunks.

2.6. What is the `ReliableSequencedPipelineStage` struct?

- Ensures that packets sent through it are delivered in order and acknowledged.

2.7. What is the `NetworkDriver's CreatePipeline()` function?

- function in `NetworkDriver` creates a new pipeline from stage types to handle data packets with various processing stages.

2.8. What is the `NetworkEndPoint` struct?

- `NetworkEndpoint` represents an endpoint on the network, typically consisting of an IP address and a port number.

2.9. What is the `NetworkEndPoint.Parse()` function?

- Parses a provided IP address and port.

2.10. What is the `NetworkDriver's ScheduleUpdate()` function?

- Function in `NetworkDriver` schedules an update job to process incoming packets and check timeouts.

-

2.11. What is the `NetworkDriver's ScheduleUpdate().Complete()` function?

- In `NetworkDriver` schedules the driver to process network events and waits for the operation to complete.

2.12. What is the `NetworkEvent.Type` enum?

- Enum represents various types of network events, such as Empty, Connect, Disconnect, and Data.

2.13. What is the `NetworkConnection's PopEvent()` function?

- Pops the next available event on the connection.

3. Learn the name of each member variable and function.
4. Read through the code, more or less, linearly. Seek to read each line and understand what it does.
5. Read through the code again and generate a list of any unanswered questions you have.
 - One main one: why would you use something like pcall system to run the stem
 - Also is the errors a unity thing or a windows network thing

Note; in some cases, an effective description of a thing can be produced by restating the name of the thing, when this is the case, it is fine to do, but if there is more to say, seek to say it.

Congratulations, transporting data across the internet is a thing you know how to do!